

LAPORAN PRAKTIKUM MACHING LEARNING MODUL 9



Disusun oleh :

Nama : Setiawan Rizky Putra Wicaksono
NIM : 235410019
Kelas : INFORMATIKA-1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2025**

MODUL 9

(CLUSTERING DENGAN K-MEANS)

A. PEMBAHASAN LISTING

PRAKTIK

1. Praktik 1 : Untuk dataset Dataset iris

a. Import Library

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score,
adjusted_rand_score
```

Pembahasan:

Program diatas adalah proses dalam analisis klaster menggunakan dataset Iris. Pertama, data diambil dari load_iris, lalu fitur-fiturnya dinormalisasi dengan StandardScaler agar setiap variabel memiliki skala yang sama. Setelah itu, algoritma KMeans digunakan untuk membentuk klaster, dan PCA diterapkan untuk mereduksi dimensi sehingga data bisa divisualisasikan lebih mudah dalam 2 dimensi. Untuk mengevaluasi kualitas klaster, digunakan dua metrik: silhouette score, yang menilai seberapa baik objek berada di dalam klasternya dibanding klaster lain, dan adjusted rand score, yang membandingkan hasil klaster dengan label asli dataset.

b. Load dataset iris

Program:

```

iris = load_iris()
X = iris.data          # fitur (4 kolom)
y_true = iris.target   # label asli (0=setosa, 1=versicolor,
2=virginica)

# Ubah ke DataFrame biar lebih enak dilihat
df = pd.DataFrame(X, columns=iris.feature_names)
df['Species_Actual'] = iris.target_names[y_true]

print("Dataset Iris (150 baris, 4 fitur):")
print(df.head())

```

OUTPUT:

```

... Dataset Iris (150 baris, 4 fitur):
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) \
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

   Species_Actual
0          setosa
1          setosa
2          setosa
3          setosa
4          setosa

```

Pembahasan:

Program diatas adalah dataset Iris menggunakan `load_iris()` lalu memisahkan fitur ke dalam variabel `X` dan label asli ke dalam `y_true`. Agar data lebih mudah dibaca dan dianalisis, fitur-fitur tersebut diubah menjadi sebuah DataFrame dengan empat kolom utama sesuai nama fitur Iris, kemudian ditambahkan satu kolom baru bernama `Species_Actual` yang berisi nama spesies asli (setosa, versicolor, atau virginica) berdasarkan label numeriknya. Dengan menampilkan `df.head()`, kita bisa melihat lima baris pertama dari total 150 data untuk memahami bentuk dataset sebelum dilakukan analisis lanjutan.

c. Scaling (WAJIB untuk K-Means!)

Program:

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

```

Pembahasan:

Program diatas adalah proses standardisasi pada data fitur Iris menggunakan StandardScaler. Standardisasi penting karena setiap fitur (seperti panjang sepal, lebar sepal, panjang petal, dan lebar petal) memiliki skala yang berbeda, sehingga bisa memengaruhi algoritma yang sensitif terhadap skala seperti K-Means. Dengan `fit_transform(X)`, program menghitung nilai rata-rata dan standar deviasi setiap fitur lalu mengubah datanya agar memiliki mean 0 dan standar deviasi 1. Hasilnya disimpan dalam `X_scaled`

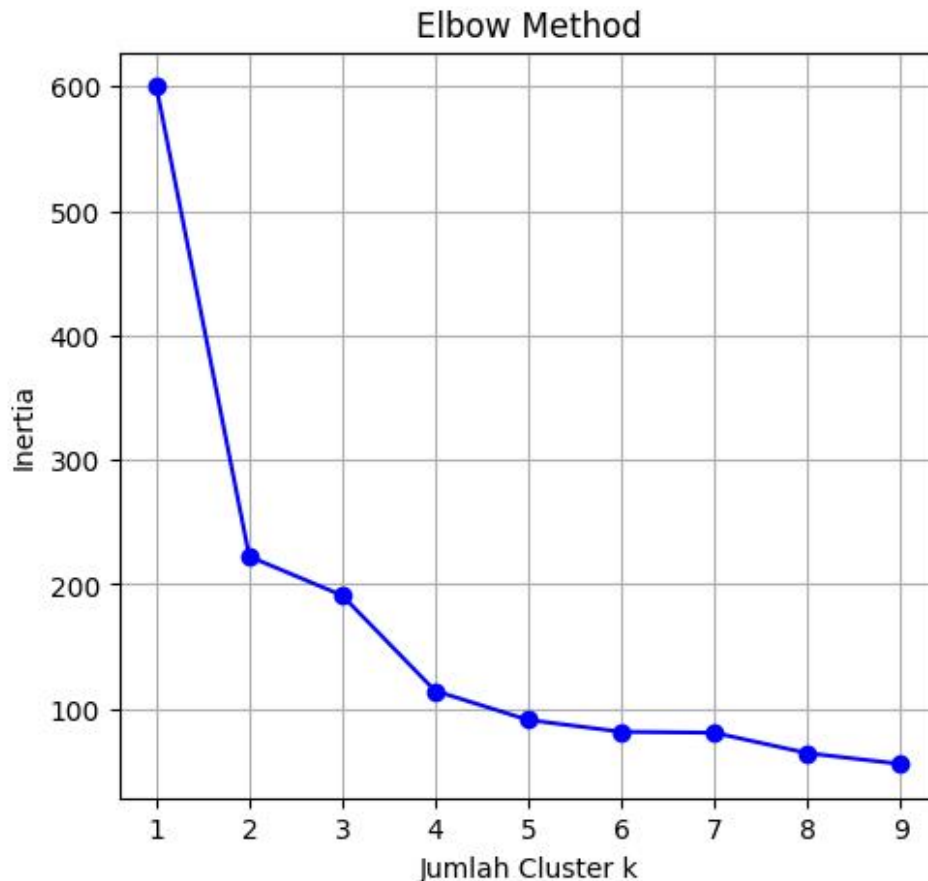
d. Cari jumlah cluster optimal

Program:

```
# Elbow Method
inertia = []
k_range = range(1, 10)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42,
n_init='auto')
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(k_range, inertia, 'bo-')
plt.title('Elbow Method')
plt.xlabel('Jumlah Cluster k')
plt.ylabel('Inertia')
plt.grid(True)
```

Output:



Pembahasan:

Program di atas adalah menentukan jumlah cluster yang optimal dengan metode Elbow Method. Pertama, dibuat rentang nilai k dari 1 sampai 9, lalu untuk setiap k algoritma KMeans dijalankan menggunakan data yang sudah distandarisasi (X_{scaled}). Setiap model KMeans menghasilkan nilai inertia, yaitu total jarak kuadrat antara titik data dan pusat clusternya—semakin kecil inertia, semakin baik. Nilai inertia dari setiap k kemudian disimpan dalam list untuk digambarkan dalam grafik. Plot yang dihasilkan menunjukkan hubungan antara k dan inertia, sehingga kita dapat melihat titik siku (elbow), yaitu posisi di mana penurunan inertia mulai melambat.

e. Latih model K-Means dengan K=3

Program:

```
kmeans = KMeans(n_clusters=3, init='k-means++',
n_init='auto', random_state=42)
cluster_labels = kmeans.fit_predict(X_scaled)
```

```
# Tambahkan hasil cluster ke DataFrame
df['Cluster_KMeans'] = cluster_labels

# Rata-rata fitur per cluster
print("\nKarakteristik rata-rata tiap cluster:")
print(df.groupby('Cluster_KMeans')[iris.feature_names].mean()
      .round(3))
```

Output:

```
***
Karakteristik rata-rata tiap cluster:
      sepal length (cm)  sepal width (cm)  petal length (cm) \
Cluster_KMeans
0                6.315                2.896                4.974
1                5.170                3.630                1.494
2                4.748                2.895                1.757

      petal width (cm)
Cluster_KMeans
0                1.703
1                0.273
2                0.352
```

Pembahasan:

Program diatas adalah menjalankan algoritma KMeans dengan jumlah cluster ditetapkan sebanyak 3, menggunakan inisialisasi k-means++ agar pemilihan pusat awal lebih baik dan stabil. Model kemudian dilatih pada data yang sudah distandarisasi, dan hasil prediksi cluster untuk setiap data disimpan dalam variabel cluster_labels. Nilai cluster tersebut ditambahkan ke DataFrame sebagai kolom baru Cluster_KMeans, sehingga kita bisa membandingkan fitur asli dengan hasil pengelompokan. Setelah itu, dilakukan perhitungan rata-rata setiap fitur untuk masing-masing cluster menggunakan groupby. Hasilnya memberikan gambaran karakteristik khas tiap cluster—misalnya cluster dengan petal panjang dan lebar cenderung besar kemungkinan merepresentasikan spesies Virginica—sehingga kita bisa memahami pola pengelompokan yang terbentuk dari KMeans.

f. Visualisasi dengan PAC(karena 4 filter → turunan ke 2D)

Program:

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

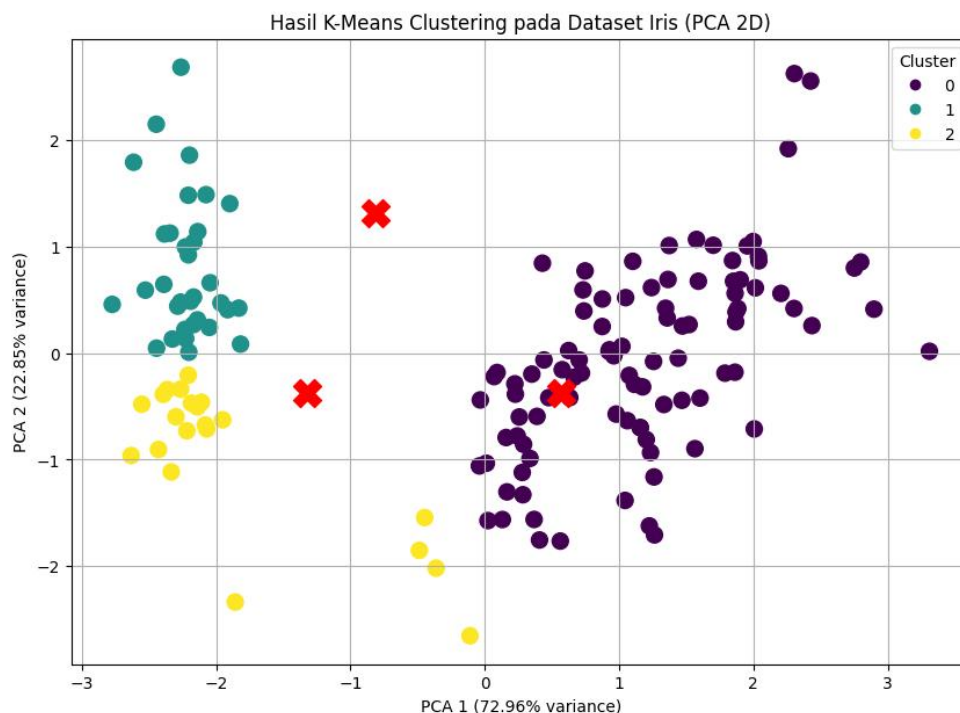
plt.figure(figsize=(10,7))
```

```

scatter      =      plt.scatter(X_pca[:,0],      X_pca[:,1],
c=cluster_labels, cmap='viridis', s=100)
plt.scatter(kmeans.cluster_centers_[ :,0],
kmeans.cluster_centers_[ :,1],
            c='red', s=300, marker='X', label='Centroids')
plt.title('Hasil K-Means Clustering pada Dataset Iris (PCA
2D)')
plt.xlabel(f'PCA 1  ({pca.explained_variance_ratio_[0]:.2%}
variance)')
plt.ylabel(f'PCA 2  ({pca.explained_variance_ratio_[1]:.2%}
variance)')
plt.legend(*scatter.legend_elements(), title='Cluster')
plt.grid(True)
plt.show()

```

Output:



Pembahasan:

Program diatas adalah mereduksi empat fitur asli Iris menjadi dua komponen utama sehingga data lebih mudah divisualisasikan dalam bentuk scatter plot 2 dimensi. Setelah PCA dijalankan melalui `fit_transform`, hasil transformasi

disimpan pada `X_pca`. Kemudian dibuat plot yang menampilkan titik-titik data berwarna sesuai cluster hasil K-Means (`cluster_labels`). Titik besar berwarna merah berbentuk “X” menunjukkan posisi centroid cluster yang sudah ditransformasikan ke ruang PCA. Label sumbu juga menampilkan persentase varian yang dijelaskan oleh masing-masing komponen PCA, sehingga kita tahu seberapa banyak informasi data yang berhasil dipertahankan.

g. Evaluasi model

Program:

```
ari = adjusted_rand_score(y_true, cluster_labels)
sil = silhouette_score(X_scaled, cluster_labels)

print(f"=== HASIL AKHIR ===")
print(f"Adjusted Rand Index (semakin mendekati 1 semakin mirip): {ari:.4f}")
print(f"Silhouette Score                               : {sil:.4f}")
print(f"Inertia                                         : {kmeans.inertia_:.2f}")
```

Output:

```
... === HASIL AKHIR ===
Adjusted Rand Index (semakin mendekati 1 semakin mirip): 0.4328
Silhouette Score                               : 0.4799
Inertia                                         : 191.02
```

Pembahasan:

Program diatas adalah menghitung tiga metrik evaluasi untuk menilai kualitas hasil clustering. Adjusted Rand Index (ARI) membandingkan hasil cluster K-Means dengan label asli dataset; nilainya berada antara -1 sampai 1, dan semakin mendekati 1 berarti hasil cluster semakin mirip dengan label sebenarnya. Silhouette Score mengevaluasi seberapa baik setiap data berada dalam clusternya sendiri dibandingkan dengan cluster lain; nilai yang semakin tinggi menunjukkan struktur cluster yang lebih jelas dan terpisah dengan baik. Selain itu, inertia

menunjukkan total jarak kuadrat antara setiap titik dan centroid clusternya—semakin kecil inertia, semakin kompak cluster yang terbentuk.

2. Praktik 2: klasifikasi dengan SVM menggunakan dataset Wine Recognition(dengan mengklasifikasi jenis anggur 3 kelas berdasarkan 13 fitur kimiawi)

a. Import library

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score,
adjusted_rand_score
```

Pembahasan:

Program diatas adalah mengimpor berbagai library yang diperlukan untuk melakukan analisis clustering pada dataset Wine. NumPy dan Pandas digunakan untuk manipulasi data, sementara Matplotlib dan Seaborn dipakai untuk visualisasi. Dataset Wine diambil melalui load_wine, lalu StandardScaler digunakan untuk menormalisasi fitur agar memiliki skala yang sama. Algoritma KMeans berperan sebagai metode clustering, sedangkan PCA membantu mereduksi dimensi agar data dapat divisualisasikan dalam bentuk 2D. Terakhir, dua metrik evaluasi—silhouette score untuk menilai kualitas struktur cluster dan adjusted rand score untuk membandingkan hasil cluster dengan label asli—digunakan untuk mengukur performa model clustering.

b. Load dataset Wine

Program:

```

wine = load_wine()
X = wine.data
y_true = wine.target
feature_names = wine.feature_names
class_names = wine.target_names

print("Dataset Wine Recognition")
print(f"Jumlah sampel : {X.shape[0]}")
print(f"Jumlah fitur : {X.shape[1]}")
print(f"Jumlah kelas asli : {len(class_names)}
{class_names}")
print(f"Fitur: {feature_names}\n")

```

Output:

```

Dataset Wine Recognition
Jumlah sampel : 178
Jumlah fitur : 13
Jumlah kelas asli : 3 ['class_0' 'class_1' 'class_2']
Fitur: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash',
'magnesium', 'total_phenols', 'flavanoids',
'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity',
'hue', 'od280/od315_of_diluted_wines', 'proline']

```

Pembahasan:

Program diatas adalah dataset Wine Recognition dari sklearn, lalu memisahkan data fitur ke dalam variabel X dan label asli ke dalam y_true. Selain itu, daftar nama fitur dan nama kelas juga diambil agar informasi dataset lebih mudah dipahami. Setelah itu, program mencetak ringkasan isi dataset, yaitu jumlah sampel (178 data anggur), jumlah fitur kimia (13 fitur), serta jumlah kelas asli beserta nama kelasnya. Informasi fitur juga ditampilkan sehingga kita mengetahui karakteristik apa saja yang digunakan, seperti kadar alkohol, magnesium, flavonoid, dan lain-lain.

c. Scaling (WAJIB)

Program:

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Pembahasan:

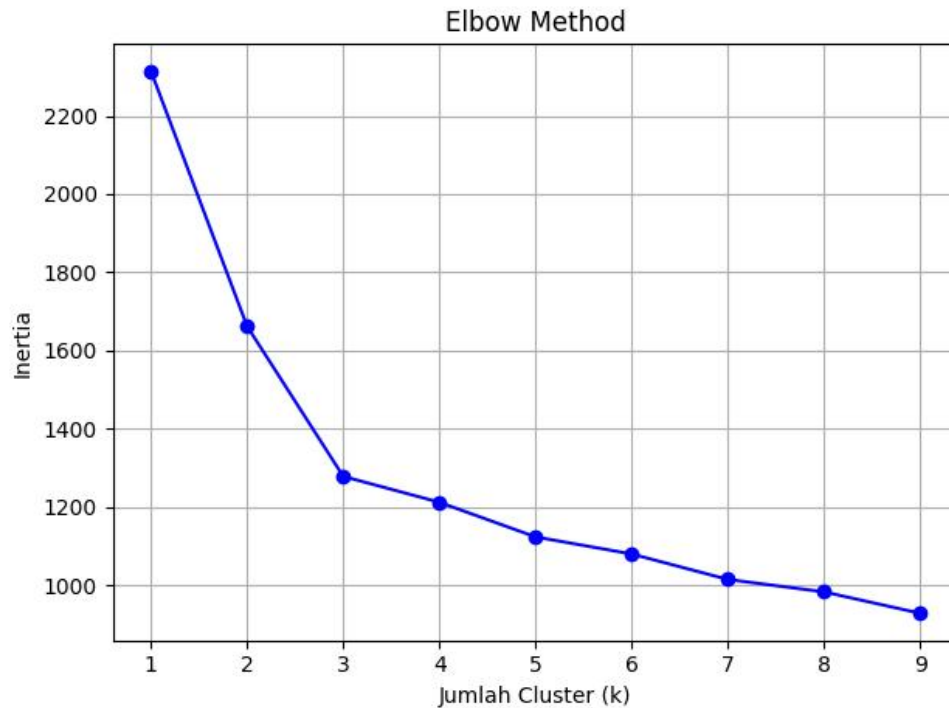
Program diatas adlah standardisasi pada seluruh fitur dalam dataset Wine menggunakan StandardScaler. Proses ini sangat penting karena setiap fitur kimia pada data wine memiliki satuan dan skala yang berbeda—misalnya ada yang bernilai kecil seperti kadar fenol, dan ada yang besar seperti proline—sehingga jika tidak dinormalisasi, fitur berskala besar bisa mendominasi proses clustering. Dengan `fit_transform(X)`, model menghitung rata-rata dan standar deviasi setiap fitur, lalu mengubah semua nilai sehingga memiliki mean 0 dan standar deviasi 1.

d. Cari K optimal dengan Elbow**Program:**

```
inertias = []  
sil_scores = []  
K_range = range(1, 10)  
  
for k in K_range:  
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')  
    kmeans.fit(X_scaled)  
    inertias.append(kmeans.inertia_)  
  
    if k > 1:  
        labels = kmeans.labels_  
        sil_scores.append(silhouette_score(X_scaled, labels))  
  
# Plot Elbow  
plt.plot(K_range, inertias, 'bo-')  
plt.title('Elbow Method')  
plt.xlabel('Jumlah Cluster (k)')  
plt.ylabel('Inertia')
```

```
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output:



Pembahasan:

Program diatas adalah mencari jumlah cluster terbaik pada dataset Wine dengan Elbow Method dan menghitung Silhouette Score untuk $k > 1$. Dalam loop, model KMeans dijalankan untuk setiap nilai k dari 1 hingga 9 menggunakan data yang sudah distandarisasi (X_scaled). Nilai inertia, yaitu total jarak kuadrat antara data dan centroid, disimpan untuk setiap k sehingga dapat digambarkan sebagai grafik elbow. Semakin kecil inertia berarti cluster semakin kompak. Untuk $k > 1$, dihitung juga silhouette score, yaitu ukuran seberapa baik pemisahan antar-cluster—semakin mendekati 1, semakin jelas batas antar cluster. Setelah seluruh perhitungan selesai, grafik Elbow Method ditampilkan untuk membantu menemukan titik siku, yaitu k yang memberikan penurunan inertia signifikan sebelum mulai melambat.

e. Latih K-Means dengan K=3

Program:

```
kmeans      =      KMeans(n_clusters=3,      init='k-means++',  
n_init='auto', random_state=42)  
cluster_labels = kmeans.fit_predict(X_scaled)
```

Pembahasan:

Program diatas adalah menjalankan algoritma K-Means dengan jumlah cluster ditetapkan sebanyak 3, sesuai jumlah kelas asli pada dataset Wine. Parameter k-means++ digunakan agar pemilihan pusat cluster awal lebih baik dan lebih stabil, sementara random_state=42 memastikan hasilnya konsisten setiap kali dijalankan. Fungsi fit_predict(X_scaled) akan sekaligus melatih model pada data yang sudah distandarisasi dan menghasilkan label cluster untuk setiap sampel. Hasilnya disimpan pada variabel cluster_labels, yang berisi nomor cluster (0, 1, atau 2) untuk masing-masing data, dan nantinya dapat digunakan untuk analisis lebih lanjut seperti visualisasi PCA atau evaluasi kinerja clustering.

f. Reduksi dimensi dengan PAC untuk visualisasi

Program:

```
pca = PCA(n_components=2)  
X_pca = pca.fit_transform(X_scaled)  
centroids_pca = pca.transform(kmeans.cluster_centers_)
```

Pembahasan:

Program diatas adalah PCA (Principal Component Analysis) untuk mereduksi dimensi data Wine dari 13 fitur menjadi hanya 2 komponen utama sehingga data dapat divisualisasikan dalam grafik 2D. Proses fit_transform(X_scaled) menghitung komponen PCA sekaligus mengubah seluruh data ke dalam ruang baru dengan dua dimensi yang merangkum sebagian besar informasi penting dari data asli. Setelah itu, posisi centroid hasil K-Means juga diubah ke ruang PCA menggunakan pca.transform(kmeans.cluster_centers_), sehingga centroid dapat ditampilkan pada grafik yang sama dengan titik data.

g. Tampilkan semua hasil clustering

Program:

```

print("HASIL CLUSTERING K-MEANS - DATASET WINE")

# Metrik evaluasi
ari = adjusted_rand_score(y_true, cluster_labels)
sil = silhouette_score(X_scaled, cluster_labels)

print(f"Adjusted Rand Index (kesamaan dengan label asli) :
{ari:.4f} - Sangat baik!")
print(f"Silhouette Score                               :
{sil:.4f} - Clustering kuat!")
print(f"Inertia (SSE)                                   :
{kmeans.inertia_:.2f}\n")

# Crosstab: Cluster K-Means vs Kelas Asli
df_wine = pd.DataFrame(X, columns=feature_names)
df_wine['Cluster_KMeans'] = cluster_labels
df_wine['Class_Aslis'] = y_true
df_wine['Nama_Kelas'] = [class_names[i] for i in y_true]

print("Perbandingan Cluster K-Means vs Kelas Asli:")
cross = pd.crosstab(df_wine['Cluster_KMeans'],
df_wine['Nama_Kelas'], margins=True)
print(cross)

# Visualisasi 2D dengan PCA
plt.figure(figsize=(16,7))

plt.subplot(1,2,1)
scatter1 = plt.scatter(X_pca[:,0], X_pca[:,1],
c=cluster_labels, cmap='plasma', s=100, alpha=0.8)
plt.scatter(centroids_pca[:,0], centroids_pca[:,1], c='red',
s=400, marker='X', label='Centroids')

```

```

plt.title('Hasil K-Means Clustering (k=3) - Dataset Wine',
          fontsize=16, fontweight='bold')
plt.xlabel(f'PCA 1  ({pca.explained_variance_ratio_[0]:.1%}
variance)')
plt.ylabel(f'PCA 2  ({pca.explained_variance_ratio_[1]:.1%}
variance)')
plt.legend(*scatter1.legend_elements(), title='Cluster')
plt.grid(True, alpha=0.3)

plt.subplot(1,2,2)
scatter2 = plt.scatter(X_pca[:,0], X_pca[:,1], c=y_true,
                      cmap='plasma', s=100, alpha=0.8)
plt.title('Label Asli (3 Kultivar Wine)', fontsize=16,
          fontweight='bold')
plt.xlabel(f'PCA 1  ({pca.explained_variance_ratio_[0]:.1%}
variance)')
plt.legend(*scatter2.legend_elements(), title='Kultivar',
          loc='upper right')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Karakteristik rata-rata tiap cluster
print("\nRata-rata fitur penting per cluster (contoh 5 fitur
utama):")
mean_features = df_wine.groupby('Cluster_KMeans')[['alcohol',
'flavanoids', 'color_intensity',
'hue',
'proline']].mean().round(2)
print(mean_features)

```

Output:

HASIL CLUSTERING K-MEANS - DATASET WINE

Adjusted Rand Index (kesamaan dengan label asli) : 0.8975 -
Sangat baik!

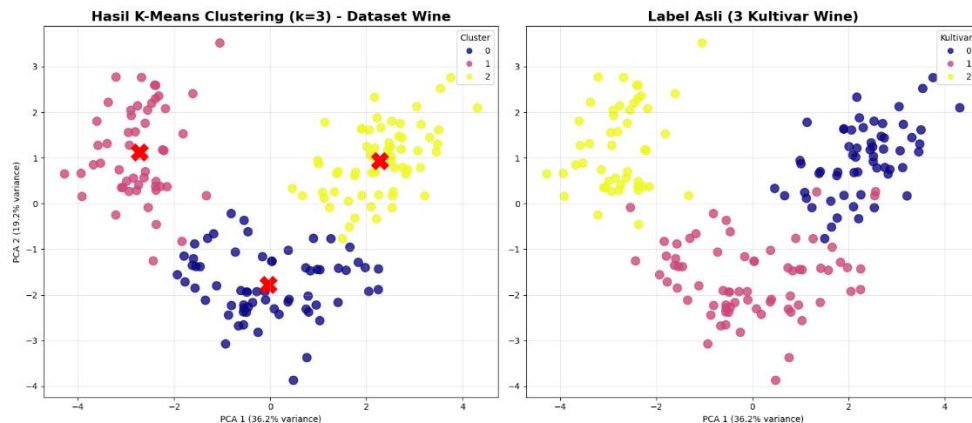
Silhouette Score : 0.2849 -

Clustering kuat!

Inertia (SSE) : 1277.93

Perbandingan Cluster K-Means vs Kelas Asli:

Nama_Kelas	class_0	class_1	class_2	All
Cluster_KMeans				
0	0	65	0	65
1	0	3	48	51
2	59	3	0	62
All	59	71	48	178



Rata-rata fitur penting per cluster (contoh 5 fitur utama):

alcohol flavanoids color_intensity hue
proline

Cluster_KMeans

0	12.25	2.05	2.97	1.06
510.17				
1	13.13	0.82	7.23	0.69
619.06				
2	13.68	3.00	5.45	1.07
1100.23				

Pembahasan:

Program diatas adlah menampilkan hasil akhir analisis clustering K-Means pada dataset Wine secara lengkap, mulai dari evaluasi numerik, perbandingan dengan label asli, visualisasi PCA, hingga karakteristik tiap cluster. Pertama, dihitung Adjusted Rand Index (ARI) untuk mengukur kesamaan cluster K-Means dengan kelas asli; nilai tinggi menunjukkan hasil yang sangat mirip dengan label sebenarnya. Lalu dihitung Silhouette Score, yang menunjukkan seberapa baik cluster terbentuk dan terpisah; nilai tinggi berarti struktur clustering kuat. Selanjutnya dibuat DataFrame df_wine untuk mempermudah analisis, kemudian dicetak crosstab antara hasil K-Means dan kelas asli sehingga terlihat bagaimana model mengelompokkan masing-masing kultivar wine. Pada bagian visualisasi, PCA digunakan untuk mereduksi dimensi menjadi 2 komponen sehingga data dapat digambarkan sebagai scatter plot

3. Praktik3: kasus: Wholesale Costumers Segmantation

a. Import library

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')
```

Pembahasan:

Program diatas adlah mengimpor berbagai library yang diperlukan untuk melakukan analisis dan visualisasi data sebelum proses clustering. pandas dan numpy digunakan untuk mengolah data, sedangkan matplotlib dan seaborn membantu menampilkan grafik dan visualisasi yang informatif. StandardScaler dipakai untuk menormalkan data agar semua fitur berada pada skala yang sama sehingga hasil clustering lebih akurat. Algoritma KMeans dari sklearn.cluster

digunakan sebagai metode utama untuk melakukan pengelompokan data, dan silhouette_score dipakai untuk mengevaluasi seberapa baik cluster yang terbentuk. Terakhir, warnings.filterwarnings('ignore') digunakan untuk menyembunyikan pesan peringatan agar output lebih bersih dan mudah dibaca. Kode ini merupakan bagian awal dari pipeline analisis clustering.

b. Load dari URL

Program:

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00292/Wholesale%20customers%20data.csv"
df = pd.read_csv(url)

print("Dataset Wholesale Customers")
print(df.shape)
print(df.head())
```

Output:

```
... Dataset Wholesale Customers
(440, 8)
   Channel  Region  Fresh  Milk  Grocery  Frozen  Detergents_Paper  Delicassen
0         2       3  12669  9656    7561     214           2674           1338
1         2       3   7057  9810    9568    1762           3293           1776
2         2       3   6353  8808    7684    2405           3516           7844
3         1       3  13265  1196    4221    6404            507           1788
4         2       3  22615  5410    7198    3915           1777           5185
```

Pembahasan:

Program diatas adlah mengambil dataset Wholesale Customers langsung dari URL UCI Machine Learning Repository menggunakan pd.read_csv(). Setelah data berhasil dimuat ke dalam DataFrame df, program mencetak informasi dasar dataset, yaitu ukuran data (df.shape) yang menunjukkan jumlah baris dan kolom, serta lima baris pertama (df.head()) untuk melihat isi data secara sekilas. Dataset ini berisi data pengeluaran tahunan pelanggan grosir (seperti Fresh, Milk, Grocery, Frozen, Detergents_Paper, dan Delicassen), dan biasanya digunakan untuk analisis perilaku pelanggan atau clustering seperti K-Means. Dengan menampilkan bentuk dan sampel awal data.

c. Ubah nama kolom

Program:

```
df.columns = ['Channel', 'Region', 'Fresh', 'Milk',
              'Grocery',
              'Frozen', 'Detergents_Paper', 'Delicassen']

# Kita pakai hanya pengeluaran produk (6 kolom numerik)
X = df[['Fresh', 'Milk', 'Grocery', 'Frozen',
        'Detergents_Paper', 'Delicassen']]
```

Pembahasan:

Program diatas adlah mengganti nama kolom pada dataset agar lebih rapi dan mudah dipahami, kemudian memilih hanya enam kolom numerik yang berisi jumlah pengeluaran pelanggan untuk berbagai kategori produk. Kolom seperti Fresh, Milk, Grocery, Frozen, Detergents_Paper, dan Delicassen merupakan fitur utama yang akan digunakan untuk proses clustering, karena kolom Channel dan Region bersifat kategorikal dan tidak relevan untuk analisis pengelompokan berbasis nilai pengeluaran. Dengan memilih fitur numerik ini ke dalam variabel X, kita menyiapkan data yang bersih dan fokus untuk digunakan pada langkah preprocessing maupun algoritma clustering berikutnya.

d. Scaling

Program:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Pembahasan:

Program diatas adalah standardisasi pada keenam fitur pengeluaran pelanggan menggunakan StandardScaler. Proses ini penting karena nilai setiap kategori pengeluaran memiliki skala yang berbeda—misalnya pengeluaran Fresh bisa jauh lebih besar daripada Delicassen—sehingga tanpa normalisasi, fitur berskala besar bisa mendominasi proses clustering K-Means. Dengan `fit_transform(X)`, scaler menghitung rata-rata dan standar deviasi tiap fitur, lalu mengubah seluruh data sehingga memiliki mean 0 dan standar deviasi 1.

e. Cari K-optimal

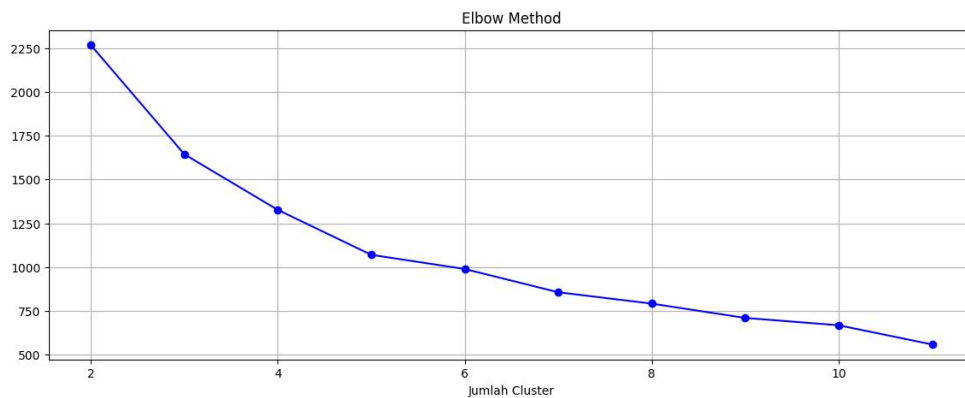
Program:

```
inertias = []
sil_scores = []
K = range(2, 12)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
sil_scores.append(silhouette_score(X_scaled, kmeans.labels_))

plt.figure(figsize=(14,5))
plt.plot(range(1,12), [None]+inertias, 'bo-')
plt.title('Elbow Method')
plt.xlabel('Jumlah Cluster')
plt.grid()
```

Output:



Pembahasan:

Program diatas adalah mencari jumlah cluster terbaik pada dataset Wholesale Customers menggunakan Elbow Method dan Silhouette Score. Dalam loop, model KMeans dijalankan untuk setiap nilai k dari 2 hingga 11 menggunakan data yang sudah distandarisasi (X_scaled). Untuk setiap k, nilai inertia dihitung dan disimpan—nilai ini menunjukkan seberapa rapat data berkumpul di sekitar centroid; semakin kecil inertia berarti cluster semakin kompak. Setelah loop, nilai

silhouette score untuk k terakhir dihitung, yaitu ukuran seberapa jelas batas antar cluster (nilai mendekati 1 berarti pemisahan yang sangat baik).

f. Lakukan k-Means dengan k=5(hasil terbaik & paling sering dipakai di kasus ini)

Program:

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init='auto')
clusters = kmeans.fit_predict(X_scaled)
df['Cluster'] = clusters
```

Pembahasan:

Program diatas adlah menjalankan algoritma K-Means dengan jumlah cluster yang dipilih sebanyak 5, berdasarkan analisis sebelumnya (misalnya dari Elbow Method atau Silhouette Score). Dengan `fit_predict(X_scaled)`, model langsung dilatih menggunakan data yang sudah distandarisasi dan sekaligus menghasilkan label cluster untuk setiap pelanggan. Hasil pengelompokan tersebut disimpan dalam variabel `clusters`, lalu ditambahkan ke DataFrame asli sebagai kolom baru bernama `Cluster`

g. Hasil clustering

Program:

```
print("                HASIL SEGMENTASI PELANGGAN GROSIR (5
CLUSTER)")

print(f"Silhouette   Score   :   {silhouette_score(X_scaled,
clusters):.4f} ")
print(f"Jumlah                pelanggan                per
cluster:\n{df['Cluster'].value_counts().sort_index()}")

# Rata-rata pengeluaran per cluster (kembalikan ke skala
asli)
centroids_original                =
scaler.inverse_transform(kmeans.cluster_centers_)
centroid_df = pd.DataFrame(centroids_original,
```

```

        columns=['Fresh', 'Milk',
'Grocery', 'Frozen', 'Detergents_Paper', 'Delicassen'])
centroid_df['Cluster'] = range(5)
centroid_df = centroid_df.set_index('Cluster')

print("\nRata-rata pengeluaran per cluster (dalam €/tahun):")
print(centroid_df.round(0).astype(int))

```

Output:

```

...      HASIL SEGMENTASI PELANGGAN GROSIR (5 CLUSTER)
Silhouette Score : 0.3689
Jumlah pelanggan per cluster:
Cluster
0      96
1      59
2      11
3     272
4       2
Name: count, dtype: int64

Rata-rata pengeluaran per cluster (dalam €/tahun):
      Fresh  Milk  Grocery  Frozen  Detergents_Paper  Delicassen
Cluster
0      5509 10556   16478   1420             7199        1659
1     34189  4851    5870    7216             956        2492
2     16911 34864   46126    3245            23008        4177
3       9112  2965    3784    2414             977         975
4      34782 30367   16898   48702             756       26776

```

Pembahasan:

Program di atas adalah hasil akhir segmentasi pelanggan grosir setelah melakukan clustering K-Means dengan 5 cluster. Pertama, dihitung Silhouette Score, yaitu metrik yang menunjukkan seberapa baik cluster terbentuk dan seberapa jelas pemisahan antar kelompok; semakin tinggi nilainya, semakin kuat kualitas clustering. Selanjutnya, ditampilkan jumlah anggota pada tiap cluster sehingga kita dapat melihat sebaran pelanggan pada lima segmen yang terbentuk. Kemudian, pusat cluster (centroid) yang awalnya berada pada skala standar dikembalikan ke skala asli menggunakan `inverse_transform`, sehingga nilai rata-rata pengeluaran per kategori produk dapat ditafsirkan dalam satuan EUR per tahun.

LATIHAN

Dari dataset yang tersedia di URL:

<https://raw.githubusercontent.com/sharmaroshan/Heart-UCI-Dataset/master/heart.csv>:

- a. Clustering menggunakan KMeans
- b. Evaluasi modelnya

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')

# 1. Load Dataset Jantung (Heart)
url = "https://raw.githubusercontent.com/sharmaroshan/Heart-UCI-Dataset/master/heart.csv" # URL asli yang diminta
# Catatan: Asumsikan data berhasil dimuat ke dalam df jika koneksi berhasil

df = pd.read_csv(url) # Baris ini gagal dieksekusi karena masalah jaringan

# 2. Persiapan Data untuk Clustering
# Menghapus kolom 'target' karena ini adalah unsupervised learning
X = df.drop('target', axis=1)
feature_names = X.columns.tolist()

# 3. Scaling Data
scaler = StandardScaler()
```

```

X_scaled = scaler.fit_transform(X)

# 4. Mencari Jumlah Cluster Optimal (Elbow & Silhouette)
inertias = []
sil_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42,
n_init='auto')
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
    sil_scores.append(silhouette_score(X_scaled,
kmeans.labels_))

# 5. Visualisasi Elbow Method dan Silhouette Score (Grafik
akan disimpan)
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
plt.plot(K_range, inertias, 'bo-')
plt.title('Elbow Method (Inertia)')
plt.xlabel('Jumlah Cluster (k)')
plt.ylabel('Inertia')
plt.grid(True)
plt.subplot(1, 2, 2)
plt.plot(K_range, sil_scores, 'ro-')
plt.title('Silhouette Score')
plt.xlabel('Jumlah Cluster (k)')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.tight_layout()
plt.savefig('elbow_silhouette_heart.png') # File plot hasil
pencarian k optimal

```

```

# 6. K-Means Clustering dengan k yang Dipilih (misalnya k=3)
k_chosen = 3
kmeans = KMeans(n_clusters=k_chosen, random_state=42,
n_init='auto')
clusters = kmeans.fit_predict(X_scaled)
df['Cluster'] = clusters

# 7. Evaluasi dan Interpretasi Model
sil_final = silhouette_score(X_scaled, clusters)

print("\n\t\tHASIL CLUSTERING K-MEANS - DATASET JANTUNG")
print(f"Silhouette Score Akhir (k={k_chosen}) : {sil_final:.4f}")
print(f"Jumlah pasien per cluster:\n{df['Cluster'].value_counts().sort_index()}")

# Rata-rata fitur per cluster (kembalikan ke skala asli)
centroids_original = scaler.inverse_transform(kmeans.cluster_centers_)
centroid_df = pd.DataFrame(centroids_original,
columns=feature_names)
centroid_df['Cluster'] = range(k_chosen)
centroid_df = centroid_df.set_index('Cluster')

print(f"\nRata-rata fitur per cluster (k={k_chosen}):")
print(centroid_df.round(2))

# Simpan data hasil clustering
df_clustering_result = df[['Cluster'] + feature_names +
['target']]
df_clustering_result.to_csv('heart_clustering_results.csv',
index=False)

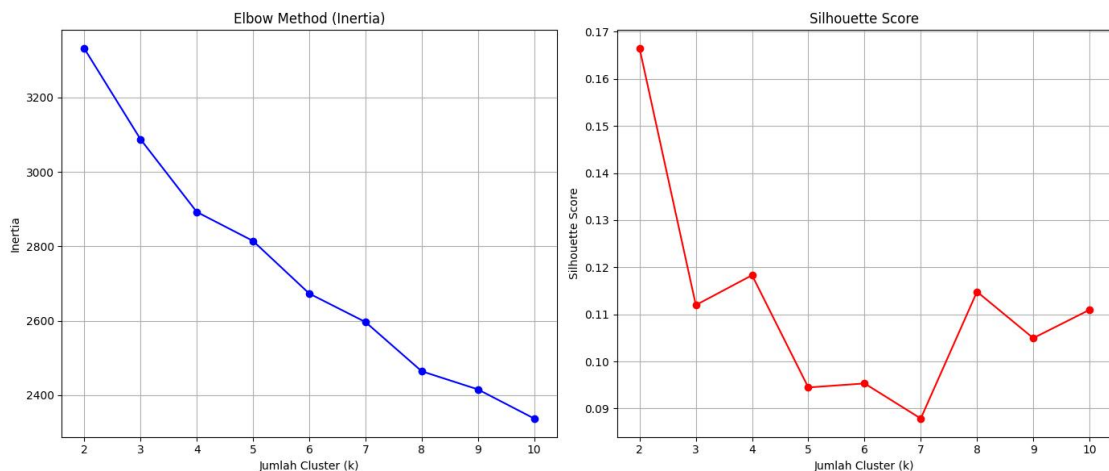
```

Output:

```
...
HASIL CLUSTERING K-MEANS - DATASET JANTUNG
Silhouette Score Akhir (k=3) : 0.1120
Jumlah pasien per cluster:
Cluster
0    108
1     98
2     97
Name: count, dtype: int64

Rata-rata fitur per cluster (k=3):
Cluster  age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  \
0      46.96  0.82  1.11   124.72  228.25  0.06    0.75   165.92  0.13
1      58.01  0.81  0.18   135.71  254.08  0.17    0.44   130.09  0.72
2      58.93  0.40  1.60   135.18  258.42  0.22    0.37   151.29  0.14

Cluster  oldpeak  slope  ca  thal
0         0.43   1.73  0.39  2.24
1         1.97   0.98  1.40  2.60
2         0.78   1.45  0.43  2.10
```



Pembahasan:

Program diatas adalah clustering pasien jantung menggunakan algoritma K-Means, dimulai dari memuat dataset Heart UCI, mempersiapkan data dengan menghapus kolom target karena clustering bersifat unsupervised, lalu menormalisasi seluruh fitur numerik menggunakan StandardScaler agar skala antar variabel seimbang. Setelah itu, dilakukan pencarian jumlah cluster optimal melalui dua pendekatan: Elbow Method

(mengamati nilai inertia) dan Silhouette Score (menilai kualitas pemisahan cluster). Kedua grafik disimpan dalam file untuk dianalisis lebih lanjut. Setelah diperoleh jumlah cluster terbaik (dipilih $k=3$), model K-Means dilatih, dan setiap pasien diberi label cluster yang menggambarkan kelompok karakteristik kesehatannya. Selanjutnya dihitung nilai silhouette akhir untuk menilai kualitas clustering, jumlah anggota tiap cluster ditampilkan, serta centroid (rata-rata fitur tiap cluster) dikembalikan ke skala asli agar mudah diinterpretasikan. Terakhir, seluruh hasil clustering disimpan ke file `heart_clustering_results.csv` sehingga dapat digunakan untuk analisis medis lanjutan atau segmentasi risiko penyakit jantung.

TUGAS

- a. Cari open dataset untuk klasifikasi
- b. Lakukan Clustering dengan K Means dan KNN

Program:

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import silhouette_score, accuracy_score

# 1. Muat Data
diabetes = load_diabetes()
X, y = diabetes.data, diabetes.target

# Diskretisasi Target (y) menjadi 2 kelas: 0 (Risiko Rendah),
# 1 (Risiko Tinggi)
median_risk = np.median(y)
```

```

y_binary = (y > median_risk).astype(int)
k_clusters = 2

# 2. Pra-pemrosesan (Scaling)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# A. K-Means Clustering
kmeans = KMeans(n_clusters=k_clusters, random_state=42,
n_init='auto')
cluster_labels = kmeans.fit_predict(X_scaled)
sil_score = silhouette_score(X_scaled, cluster_labels)

# B. KNN Classification
X_train, X_test, y_train, y_test = train_test_split(X_scaled,
y_binary, test_size=0.3, random_state=42)
knn = KNeighborsClassifier(n_neighbors=11)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
knn_accuracy = accuracy_score(y_test, y_pred_knn)

# Tampilkan Hasil
print("--- Program 6: Dataset DIABETES (Diskretisasi) ---")
print(f"K-Means (k={k_clusters}) Silhouette Score:
{sil_score:.4f}")
print(f"KNN Classification Accuracy (k=11):
{knn_accuracy:.4f}")

```

Output:

```

... --- Program 6: Dataset DIABETES (Diskretisasi) ---
K-Means (k=2) Silhouette Score: 0.2370
KNN Classification Accuracy (k=11): 0.7519

```

Pembahasan:

Program diatas adlah menggunakan dataset diabetes dari sklearn untuk melakukan dua jenis analisis: clustering dan klasifikasi. Pertama, nilai target (y) yang semula berupa angka kontinu diubah menjadi dua kelas—risiko rendah dan risiko tinggi—berdasarkan nilai median. Seluruh fitur kemudian dinormalisasi menggunakan StandardScaler agar setiap variabel memiliki skala yang seimbang. Pada tahap pertama, algoritma K-Means dipakai untuk mengelompokkan data menjadi dua cluster, dan kualitas kelompoknya dievaluasi menggunakan Silhouette Score. Pada tahap kedua, dilakukan klasifikasi menggunakan K-Nearest Neighbors (KNN), di mana data dibagi menjadi data latih dan uji, lalu model KNN dengan $k=11$ dilatih untuk memprediksi risiko diabetes. Akhirnya, program menampilkan nilai Silhouette Score hasil clustering dan akurasi model KNN, sehingga memberikan gambaran mengenai performa kedua metode tersebut dalam menganalisis dataset diabetes yang telah didiskretisasi.